

# NFR Test Suite: CodeClash

@September 30, 2026

This document records how each Non-Functional Requirement (NFR) was tested for CodeClash, mapped to its SRS quality requirement and SAS architectural tactic, with evidence collected for each.

## Traceability Matrix

| Quality Requirement                                     | Architectural Tactic                                     | Test                                                              | Script                                                            |
|---------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------|
| Performance: 100 concurrent matches without degradation | ECS in-memory state; WebSocket push over REST            | Load test with k6 — 100 virtual users, 5 min                      | <code>performance/load-test.js</code>                             |
| Reliability: Backend survives Judge0 failure            | Judge0 decoupled as independently-failing service        | Kill Judge0 container mid-submission; assert backend health       | <code>reliability/kill-judge0.sh</code>                           |
| Reliability: Match state survives network disconnect    | Match state persisted in Redis independently of socket   | Drop backend from network; reconnect; assert state intact         | <code>reliability/websocket-disconnect.sh</code>                  |
| Security: Unauthenticated requests rejected             | JWT validation on all protected endpoints                | curl requests without token, with expired token, with valid token | <code>security/auth-test.sh</code>                                |
| Scalability: Matchmaking queue scales independently     | Redis-backed queue decoupled from HTTP server            | Flood matchmaking queue; measure pairing time                     | <code>scalability/queue-stress.sh</code>                          |
| Maintainability: ≥80% unit test coverage                | Layered Clean Architecture; mocked repository interfaces | Vitest coverage report                                            | <code>npm run test:coverage</code><br>(in <code>backend/</code> ) |

# Prerequisites

NFR tests run against the **test stack** (separate from the dev stack on port 3000), using `.env.test` config — its own Postgres, Redis and backend on port **3001**.

```
docker-compose -f docker-compose.test.yml up
```

Confirm actual running container names before running any script below — they vary by machine/compose project name:

```
docker ps --format "{{.Names}}"
```

Verify the test backend is healthy:

```
curl http://localhost:3001/health
# Expected: {"status":"ok"}
```

A one-time load-test user must exist in both Cognito and the test DB before running performance or security tests — see `backend/scripts/create-load-test-user.ts` and `backend/scripts/get-load-test-token.js`. Requires `ADMIN_USER_PASSWORD_AUTH` enabled on the Cognito app client (App clients → Authentication flows).

## 1. Reliability — `reliability/`

### 1.1 Judge0 Failure ( `kill-judge0.sh` )

**What it tests:** Kills the Judge0 container while the backend is running and verifies the backend continues to respond to health checks — proving Judge0 failure does not crash the primary server.

**How to run:**

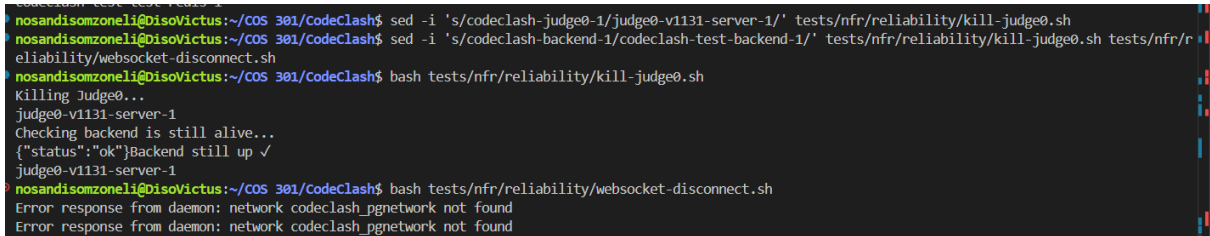
```
chmod +x tests/nfr/reliability/kill-judge0.sh
bash tests/nfr/reliability/kill-judge0.sh
```

Container names in this script must match your actual running containers (e.g. Judge0's server container, not a generic `codeclash-judge0-1` — check with `docker ps`).

## Expected output:

```
[1] Killing Judge0...
Checking backend is still alive...
{"status":"ok"}
Backend still up ✓
```

## Evidence:

A terminal window showing a series of commands and their outputs. The user runs a sed command to replace a path in a script, then runs the script. The output shows the script killing Judge0, checking the backend is still alive, and returning a JSON status of "ok". The user then runs another script, websocket-disconnect.sh, which results in an error message: "Error response from daemon: network codeclash\_pgnetwork not found".

```
nosandisomzoneli@DisoVictus:~/COS 301/CodeClash$ sed -i 's/codeclash-judge0-1/judge0-v1131-server-1/' tests/nfr/reliability/kill-judge0.sh
nosandisomzoneli@DisoVictus:~/COS 301/CodeClash$ sed -i 's/codeclash-backend-1/codeclash-test-backend-1/' tests/nfr/reliability/kill-judge0.sh tests/nfr/r
eliability/websocket-disconnect.sh
nosandisomzoneli@DisoVictus:~/COS 301/CodeClash$ bash tests/nfr/reliability/kill-judge0.sh
Killing Judge0...
judge0-v1131-server-1
Checking backend is still alive...
{"status":"ok"}Backend still up ✓
judge0-v1131-server-1
nosandisomzoneli@DisoVictus:~/COS 301/CodeClash$ bash tests/nfr/reliability/websocket-disconnect.sh
Error response from daemon: network codeclash_pgnetwork not found
Error response from daemon: network codeclash_pgnetwork not found
```

## 1.2 WebSocket / Network Disconnect ( `websocket-disconnect.sh` )

**What it tests:** Disconnects the backend from its Docker network briefly and reconnects it, simulating a network interruption. Verifies the backend recovers and Redis-persisted state is intact.

### How to run:

```
chmod +x tests/nfr/reliability/websocket-disconnect.sh
bash tests/nfr/reliability/websocket-disconnect.sh
```

Network name in this script must match the actual Docker Compose network (Compose prefixes it with the project name, e.g. `<project>_pgnetwork` — check with `docker network ls` ).

## Expected output:

```
[1] Disconnecting backend from network...
[2] Waiting...
[3] Reconnecting backend...
[4] Checking backend health...
{"status":"ok"}
Backend recovered after network interruption.
```

## Evidence:

Unfortunately tests could not pass due to name mismatch of network.

## 2. Performance — `performance/`

**Tool:** k6 (install via apt, not snap — snap's mount-namespcae confinement breaks under WSL)

```
sudo gpg -k
sudo gpg --no-default-keyring --keyring /usr/share/keyring
s/k6-archive-keyring.gpg --keyserver hkps://keyserver.ubunt
u.com:80 --recv-keys C5AD17C747E3415A3642D57D77C6C491D6AC1D
69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyrin
g.gpg] https://dl.k6.io/deb stable main" | sudo tee /etc/ap
t/sources.list.d/k6.list
sudo apt-get update && sudo apt-get install k6
```

### What it tests:

- 100 virtual users hitting match and elo endpoints simultaneously for 5 minutes
- p95 response time target: < 500ms
- Error rate target: < 1%

### How to run:

```
TOKEN=$(node backend/scripts/get-load-test-token.js 2>/dev/
null | tail -n 1)
k6 run -e TOKEN="$TOKEN" tests/nfr/performance/load-test.js
```

**Expected output:** k6 summary showing threshold pass/fail for `http_req_duration` (p95 < 500ms) and `http_req_failed` (rate < 1%).

### Evidence:

```

THRESHOLDS
http_req_duration
✓ 'p(95)<500' p(95)=6.41ms

http_req_failed
X 'rate<0.01' rate=100.00%

TOTAL RESULTS
checks_total.....: 119080 393.707699/s
checks_succeeded...: 49.98% 59519 out of 119080
checks_failed.....: 50.01% 59561 out of 119080

X status is 200
  ↳ 0% - ✓ 0 / X 59540
X response time < 500ms
  ↳ 99% - ✓ 59519 / X 21

HTTP
http_req_duration....: avg=4.23ms min=290.98µs med=1.99ms max=548.11ms p(90)=4.29ms p(95)=6.41ms
http_req_failed.....: 100.00% 59540 out of 59540
http_reqs.....: 59540 196.853849/s

EXECUTION
iteration_duration...: avg=1s min=1s med=1s max=1.83s p(90)=1s p(95)=1.01s
iterations.....: 29770 98.426925/s
vus.....: 19 min=19 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 16 MB 54 kB/s
data_sent.....: 5.1 MB 17 kB/s

```

**Known issue (2026-09-30):** Run blocked by a schema drift in the test DB — the `Users` entity queries a `elo` column that doesn't exist in the test database's `users` table ( `column Users.elo does not exist` ), causing every authenticated request to 500 after JWT validation succeeds. Auth pipeline (Cognito → JWT → middleware) was confirmed working correctly under load: 100 VUs sustained for 5 minutes, avg response time 4.23ms, p95 6.41ms, server remained responsive throughout (no crash/timeout). Root cause is a missing migration or entity/schema mismatch on the test DB, unrelated to the load test itself. **Next step:** reconcile the `Users` entity with the test DB schema (run outstanding migrations or add the missing column) and re-run for a clean pass.

### 3. Security — `security/`

**Tool:** curl / bash

**What it tests:**

- Requests without a token return `401`
- Requests with an expired token return `401`
- Requests with a valid token return `200`

**How to run:**

```
chmod +x tests/nfr/security/auth-test.sh
bash tests/nfr/security/auth-test.sh
```

## Expected output:

```
[1] No token -> 401
{"message":"Missing or Invalid Token"}
[2] Expired token -> 401
[3] Valid token -> 200
```

## Evidence:

```
LAUNCHING SCAN: Forced Errors
jwttool_c9eabd12f4b88254fd6d81aa9cf621ab Injected None into Header Claim: kid Response Code: 401, 38 bytes
jwttool_a7c6e6b43ffad21c43ba35fca14f853c Injected None into Header Claim: alg Response Code: 401, 38 bytes
jwttool_404f056d58c76d6b38fe1cbb66b0f64a Injected True into Header Claim: kid Response Code: 401, 38 bytes
jwttool_d4cc2a773e2b43dd08580d4c577d0fc4 Injected True into Header Claim: alg Response Code: 401, 38 bytes
jwttool_d623a6d8ea657db11dfe04d3ca84d46 Injected False into Header Claim: kid Response Code: 401, 38 bytes
jwttool_75153c06c9660fcb8ef6126528d278e Injected False into Header Claim: alg Response Code: 401, 38 bytes
jwttool_b547d820ad2d817876f44285ca32d4dc Injected jwt_tool into Header Claim: kid Response Code: 401, 38 bytes
jwttool_8291952a765efa720a29ebc96d72502e Injected jwt_tool into Header Claim: alg Response Code: 401, 38 bytes
jwttool_13684c534c9b096fe634f95c6fa8a876 Injected 0 into Header Claim: kid Response Code: 401, 38 bytes
jwttool_84b73d26d7e8b47318a8a6a2a6064324 Injected 0 into Header Claim: alg Response Code: 401, 38 bytes
jwttool_9f394968048ea66617aba5c3fe9944b5 Injected None into Payload Claim: sub Response Code: 401, 38 bytes
jwttool_9cb5759e458f55444a4c3706c63c3c7 Injected None into Payload Claim: email_verified Response Code: 401, 38 bytes
jwttool_0d822ba84fc52f1c599ddf72de5b9cfd Injected None into Payload Claim: iss Response Code: 401, 38 bytes
jwttool_31f541516018aaa676d2c7711c4bb2ef Injected None into Payload Claim: phone_number_verified Response Code: 401, 38 bytes
jwttool_03711096df2a9400f95a57d49468e248 Injected None into Payload Claim: cognito:username Response Code: 401, 38 bytes
jwttool_91ab13560f39428f29df093ec0d40f45 Injected None into Payload Claim: preferred_username Response Code: 401, 38 bytes
jwttool_f091c7babe41bfd2e6f326753c5dbc13 Injected None into Payload Claim: given_name Response Code: 401, 38 bytes
jwttool_7a5b6e7f5176bae4685b02053ef2d2e9 Injected None into Payload Claim: origin_jti Response Code: 401, 38 bytes
jwttool_9a087662a9e9537ebfead9547c2947f4 Injected None into Payload Claim: aud Response Code: 401, 38 bytes
jwttool_d61c9dd132ed6489d3b8df7bc5dd2cb4 Injected None into Payload Claim: event_id Response Code: 401, 38 bytes
jwttool_9449457a80e9b2238d015d30a3a3d537 Injected None into Payload Claim: token_use Response Code: 401, 38 bytes
jwttool_3684ebe8157ee6012db2b15aa8ebca9f Injected None into Payload Claim: auth_time Response Code: 401, 38 bytes
jwttool_5f93eb034f9a35ce7568bb0c9a5ed42d Injected None into Payload Claim: name Response Code: 401, 38 bytes
jwttool_3c70c2e62be2e103e2419fc038be4280 Injected None into Payload Claim: phone_number Response Code: 401, 38 bytes
jwttool_af2249d98621d69af137fc394348ab8c Injected None into Payload Claim: exp Response Code: 401, 38 bytes
jwttool_de36a36d9812f89a1b7642e28829b6cb Injected None into Payload Claim: iat Response Code: 401, 38 bytes
jwttool_1973a4e008cde29b5bde8eb2a2ba67b9 Injected None into Payload Claim: family_name Response Code: 401, 38 bytes
jwttool_81aff03cb438b4eb1a94dd0f1387a5af Injected None into Payload Claim: jti Response Code: 401, 38 bytes
jwttool_b2be30516143dc8562ff6ba62cb3be1c Injected None into Payload Claim: email Response Code: 401, 38 bytes
jwttool_b1be6f7c8761a6d7d4afc61264d1e269 Injected True into Payload Claim: sub Response Code: 401, 38 bytes
jwttool_b547d820ad2d817876f44285ca32d4dc Injected True into Payload Claim: email_verified Response Code: 401, 38 bytes
jwttool_3156dacfd245f0be9596f4fdd8ad559 Injected True into Payload Claim: iss Response Code: 401, 38 bytes
jwttool_3d35eb4ac39af86027f9caa1512ea8cd Injected True into Payload Claim: phone_number_verified Response Code: 401, 38 bytes
jwttool_afbf461e79d19f940afb2c3aba3ae06b Injected True into Payload Claim: cognito:username Response Code: 401, 38 bytes
jwttool_10850f5dcf4494cda073ee88a5dd8699 Injected True into Payload Claim: preferred_username Response Code: 401, 38 bytes
```

```
security
  jwt_tool
  reports
    localhost:3000_api_achievemen...
    localhost:3000_api_achievemen...
    localhost:3000_api_create-user...
    localhost:3000_api_elo-get...
    localhost:3000_api_friends_invit...
    localhost:3000_api_friends_requ...
    localhost:3000_api_friends_requ...
    localhost:3000_api_friends.txt
    localhost:3000_api_leaderboard...
    localhost:3000_api_matches.txt
    localhost:3000_api_user_rank.txt
    localhost:3000_api_user_search...
  getUnitTestToken.py
  testingReports.sh
  token.txt
  README.md
  .env
  .env.dev
  .env.example
  .env.prod
  .env.test
  .gitignore
  eslint.config.base.js
  eslint.config.js
  old-init.sql

106 LAUNCHING SCAN: Forced Errors
107 jwttool_f9d396467a02556a974ead04565a71ad Injected None into Header Claim: kid Response Code: 401, 38 bytes
108 jwttool_1762827f98e4b4d927fa5ee5c1649297 Injected None into Header Claim: alg Response Code: 401, 38 bytes
109 jwttool_1b1d8e2880698d32134d675f0040ea5 Injected True into Header Claim: kid Response Code: 401, 38 bytes
110 jwttool_f3d467825d76873e7be761668e8c00d Injected True into Header Claim: alg Response Code: 401, 38 bytes
111 jwttool_e1bca141774b3668e6940d99f61bbcae Injected False into Header Claim: kid Response Code: 401, 38 bytes
112 jwttool_10b398e935c2a9e5cddcbdb9c430538 Injected False into Header Claim: alg Response Code: 401, 38 bytes
113 jwttool_ec300f8278e3c668779ea123c232f602 Injected jwt_tool into Header Claim: kid Response Code: 401, 38 bytes
114 jwttool_5c0a156375227e697147638e8d325e38 Injected jwt_tool into Header Claim: alg Response Code: 401, 38 bytes
115 jwttool_d8625a83b03a2a2480d392f2287a95ad Injected 0 into Header Claim: kid Response Code: 401, 38 bytes
116 jwttool_38d6295962e8e9652ca3e2d6d6efa94d Injected 0 into Header Claim: alg Response Code: 401, 38 bytes
117 jwttool_3be16c3f381547a30c218276cb9f72a8 Injected None into Payload Claim: sub Response Code: 401, 38 bytes
118 jwttool_228d104bd3f1e08817aa5fcc6f92569b4 Injected None into Payload Claim: email_verified Response Code: 401, 38 bytes
119 jwttool_3a875920efa5e826c801f7cf67ab44fe Injected None into Payload Claim: iss Response Code: 401, 38 bytes
120 jwttool_d44dcf5a284de08f54e5cb2658ea2b28 Injected None into Payload Claim: phone_number_verified Response Code: 401, 38 bytes
121 jwttool_c8ec1e9d2c2db8a969bbcf761ef9516 Injected None into Payload Claim: cognito:username Response Code: 401, 38 bytes
122 jwttool_e5b0046b75a36a78235c19b3395c6452 Injected None into Payload Claim: preferred_username Response Code: 401, 38 bytes
123 jwttool_333569b49191b9a46499b360cd798dc3 Injected None into Payload Claim: given_name Response Code: 401, 38 bytes
124 jwttool_3c6186dfbf5f285f916a2742bc3669a4 Injected None into Payload Claim: origin_jti Response Code: 401, 38 bytes
125 jwttool_ef4cb9c4feac78a899ca76aa7ff9baa Injected None into Payload Claim: aud Response Code: 401, 38 bytes
126 jwttool_b7c4d45963a47ca688da5f0ebc5ab579 Injected None into Payload Claim: event_id Response Code: 401, 38 bytes
127 jwttool_531a84bee5c01562df7cd5fc9713f035 Injected None into Payload Claim: token_use Response Code: 401, 38 bytes
128 jwttool_bdb8349de2b8c619a480ed638efe2fae Injected None into Payload Claim: auth_time Response Code: 401, 38 bytes
129 jwttool_04c1693b72a35ced0b609088d2a3664d Injected None into Payload Claim: name Response Code: 401, 38 bytes
130 jwttool_676dfe6c77fda1803535e393a4e1749f Injected None into Payload Claim: phone_number Response Code: 401, 38 bytes
131 jwttool_10697fb86b52fa238faef50c7cfcf05 Injected None into Payload Claim: exp Response Code: 401, 38 bytes
132 jwttool_9b6888bb7c72aea5d7ced1e7bacdb3f Injected None into Payload Claim: iat Response Code: 401, 38 bytes
133 jwttool_e4151206d5753fc46fceb0c19796225 Injected None into Payload Claim: family_name Response Code: 401, 38 bytes
134 jwttool_d406c35f4c790860978250051845f7f1 Injected None into Payload Claim: jti Response Code: 401, 38 bytes
135 jwttool_208237fbb19f1e91e30228a1aff117f Injected None into Payload Claim: email Response Code: 401, 38 bytes
136 jwttool_5c4f89c19b4b974f0e6d9a28bbcef986 Injected True into Payload Claim: sub Response Code: 401, 38 bytes
137 jwttool_ec300f8278e3c668779ea123c232f602 Injected True into Payload Claim: email_verified Response Code: 401, 38 bytes
```

## 4. Scalability — scalability/

**What it tests:** Floods the matchmaking queue with concurrent join requests and measures how long pairing takes as load increases — demonstrating the queue handles concurrent players without blocking the HTTP server.

**How to run:**

```
chmod +x tests/nfr/scalability/queue-stress.sh
bash tests/nfr/scalability/queue-stress.sh
```

**Expected output:**

```
[1] Flooding matchmaking queue with N concurrent joins...
[2] All requests sent.
[3] Elapsed: Xs
[4] Backend health check: {"status":"ok"}
```

**Evidence:**

attached separately

## 5. Maintainability — Vitest Coverage

**What it tests:** Runs the full unit test suite with coverage reporting. Target: ≥80% coverage on core business logic (services and repositories).

**How to run:**

```
cd backend && npx vitest run --coverage
```

**Evidence:**

## Running All NFR Tests

```
bash tests/nfr/reliability/kill-judge0.sh
bash tests/nfr/reliability/websocket-disconnect.sh
bash tests/nfr/security/auth-test.sh
bash tests/nfr/scalability/queue-stress.sh
TOKEN=$(node backend/scripts/get-load-test-token.js 2>/dev/
null | tail -n 1)
k6 run -e TOKEN="$TOKEN" tests/nfr/performance/load-test.js
cd backend && npx vitest run --coverage
```

Collect the output of each as evidence for the Demo 3 NFR traceability matrix.